

何时该让大模型来做图节点分类？一篇系统实证分析的中文精读 导读

原文：Wu, Shen, Ge, Shan, Jiao, Sun, Cheng (ICML 2025)

本导读为面向初学者的中文学习笔记：含要点翻译、框架图解读与关键结论数据重述。

论文卡片

标题	When Do LLMs Help With Node Classification? A Comprehensive Analysis
作者	Xixi Wu, Yifei Shen, Fangzhou Ge, Caihua Shan, Yizhu Jiao, Xiangguo Sun, Hong Cheng
机构	香港中文大学、微软亚洲研究院、伊利诺伊大学厄巴纳-香槟分校
会议	ICML 2025 (国际机器学习大会)
arXiv	2502.00829
代码	https://llmnodebed.github.io/
方向	图 (Graph) × 大模型 × 系统性实证评测 (Benchmark)
一句话	不提新方法，而是把各种“用 LLM 做图节点分类”的套路拉到统一框架里公平比一遍，告诉你什么时候、用哪招才真有用。

1 一、这篇论文要回答什么问题

最近两年，大家都在尝试用大语言模型 (LLM, Large Language Model, 如 GPT-4o、LLaMA) 去做一件传统上由图神经网络 (GNN) 负责的事情——节点分类 (给图里的每个点打标签，比如判断一篇论文属于哪个学科、一个用户是不是水军)。论文很多，效果看起来都不错，但作者一针见血地指出一个尴尬现状：**缺乏清晰的设计指南**。

具体来说，工程师在真实项目里会犯难：

1. LLM 又慢又贵 (训练、推理成本远高于传统 GNN)。到底**什么情况下**多花这份钱才值得？
2. “用 LLM 做节点分类”有好多**流派** (后面会讲到 Encoder/Explainer/Predictor 等)，每一派各自适合什么场景？

3. 已有的对比工作（如 GLBench）因为**骨干模型不统一、代码库不统一**，比出来的结论**不公平、不可靠**，没法直接拿来指导选型。

本文目标：不发明新方法，而是做一次**公平、系统、可复现**的大比武。为此作者（1）造了一个统一的测试平台 **LLMNodeBed**；（2）训练并评测了**超过 2,700 个模型**，最终提炼出 **8 条可操作的经验 (Takeaway)**，直接回答“何时该上 LLM、上哪一派”。这是一篇典型的**分析类/基准类 (benchmark)** 工作，对工程选型极有参考价值。

2 二、摘要中译

节点分类是图分析中的一项基础任务，在众多领域有广泛应用。近来大语言模型 (LLM) 的突破，催生了一批基于 LLM 的节点分类方法。尽管许多研究都展示了 LLM 方法令人瞩目的性能，但**缺乏清晰的设计指南**可能会阻碍它们在实践中的落地。本文旨在通过对这些算法进行 **公平、系统的对比**来建立这样的指南。作为第一步，我们开发了 **LLMNodeBed**——一个用于“基于 LLM 的节点分类”的综合代码库与测试平台。它包含 10 个同质图 (homophilic) 数据集、4 个异质图 (heterophilic) 数据集、8 种基于 LLM 的算法、8 个经典基线，以及 3 种学习范式。随后，我们开展了大规模实验，**训练并评测了超过 2,700 个模型**，以确定影响性能的关键设定（如学习范式、同质性）与关键组件（如模型规模、提示词）。我们的发现总结出 **8 条洞见**，例如：（1）在**半监督**设定下，基于 LLM 的方法能显著超过传统方法，但在**监督**设定下这种优势变得微弱；（2）在**零样本**设定下，图基础模型 (GFM) 能击败开源 LLM，但仍不及 GPT-4o 这类强 LLM。我们希望 LLMNodeBed 的发布连同这些洞见，能推动可复现研究并启发该领域的后续工作。代码与数据集发布于 <https://llmnodebed.github.io/>。

3 三、核心贡献与三句话结论

三大贡献：

1. **一个测试平台**：发布 LLMNodeBed（基于 PyG），统一了骨干模型与代码，初版含 14 个数据集、8 种 LLM 算法、3 种学习范式，一条命令即可跑全部实验、自动生成论文里所有表格。
2. **一次大规模公平实验**：训练评测 > 2,700 个模型，系统分析了学习范式、同质性、语言模型类型与规模、提示词设计如何影响各类算法。
3. **8 条洞见与实操建议**：明确指出每一类算法在什么设定下最强、关键组件怎么选，给出直观解释与“避坑”提示。

如果只记三句话：

1. 标签少（半监督）时，LLM 真香；标签多（监督）时，老老实实用 GNN 就够了，LLM 的提升变得很小、未必划算。
2. 图“没什么用”时（异质图、结构信息弱），LLM 当编码器优势巨大；图本身信息很强时，换不换 LLM 差别不大。
3. 零样本时，把图结构喂给强 LLM（如 GPT-4o）直接推理效果最好；专门训练的图基础模型还追不上它。

4 四、预备知识（小白先看这里）

【小白补丁】什么是节点分类（Node Classification）

一张图由**节点**（点，代表实体，如论文、用户、商品）和**边**（连线，代表关系，如引用、关注、共同购买）组成。节点分类就是：已知一部分节点的标签，去**预测其余节点的标签**。本文研究的是**文本属性图（TAG, Text-Attributed Graph）**——每个节点还自带一段文字（如论文的标题摘要、商品的描述），正是这段文字给了 LLM 用武之地。

【小白补丁】三种“用 LLM”的范式：Encoder / Explainer / Predictor

这是全文最重要的分类，务必记住（对应图 1）：

- **LLM-as-Encoder（当编码器）**：让 LLM 把每个节点的文字编码成**更好的特征向量**，再喂给一个 GNN 去分类。LLM 只负责“把文字变成好用的数字”。代表方法 ENGINE。
- **LLM-as-Explainer（当解释器/增强器）**：让 LLM 用生成能力给节点文字“**扩写、补充解释**”（例如先让它预测 + 给理由），把原文本变丰富，再走“编码 + GNN”。代表方法 TAPE。
- **LLM-as-Predictor（当预测器）**：把图结构信息也塞进 LLM，让 LLM **直接吐出标签**。代表方法 GraphGPT、LLaGA，以及纯指令微调 LLM（本文记作 LLM_{IT}）。

通俗记忆：Encoder “只出特征”、Explainer “先扩写文本再出特征”、Predictor “一条龙直接出答案”。

【小白补丁】什么是同质性（Homophily）——本文的关键变量

同质性 = “物以类聚”的程度，指一条边两端的节点有多大概率属于同一类。同质图（homophilic）：相连的点大多同类（如论文引用网络，引用关系常发生在同领域论文间，本文中 Cora 同质率 82.5%）。

此时图结构本身就是强信号，GNN 沿着边传播就能分对很多。异质图 (heterophilic)：相连的点常常不同类（本文中 Texas 等网页数据集同质率仅 6.7%）。此时图结构帮不上忙，甚至会误导，模型更要依赖节点自身的文字内容——这正是 LLM 编码器的强项。记住一句话：图越“没用”（同质性越低），LLM 越能帮上忙。

【小白补丁】三种学习设定与零样本里的两条路

半监督 (Semi-supervised)：只给极少带标签的节点（如每类几个），考验“少标签也能学好”。

监督 (Supervised)：给大量带标签节点（本文多数数据集用 60% 训练）。

零样本 (Zero-shot)：完全不给目标图的标签去训练。零样本下又分两条路：(a) **Direct Inference 直接推理**——精心设计提示词 (Prompt) 让 LLM 当场分类；(b) **GFM (图基础模型, Graph Foundation Model)**——先在别的大图上预训练好，再迁移到目标图。

形式化记号（看不懂可跳过）：文本属性图记作 $G = (V, E, S)$ ，其中 V 是节点集、 E 是边集、 S 是每个节点 v 携带的文字。有标签节点记 V_l 、无标签节点记 V_u 。传统做法把文字用 bag-of-words 或 TF-IDF 压成“浅层嵌入” $X \in \mathbb{R}^{|V| \times d}$ 喂给 GNN；而 LLM 方法直接吃原始文字，指望预训练知识带来增益。

5 五、实验设计与数据集（怎么做到“公平”）

作者用图 1 这张总览图把整个测试平台 LLMNodeBed 讲清楚了。

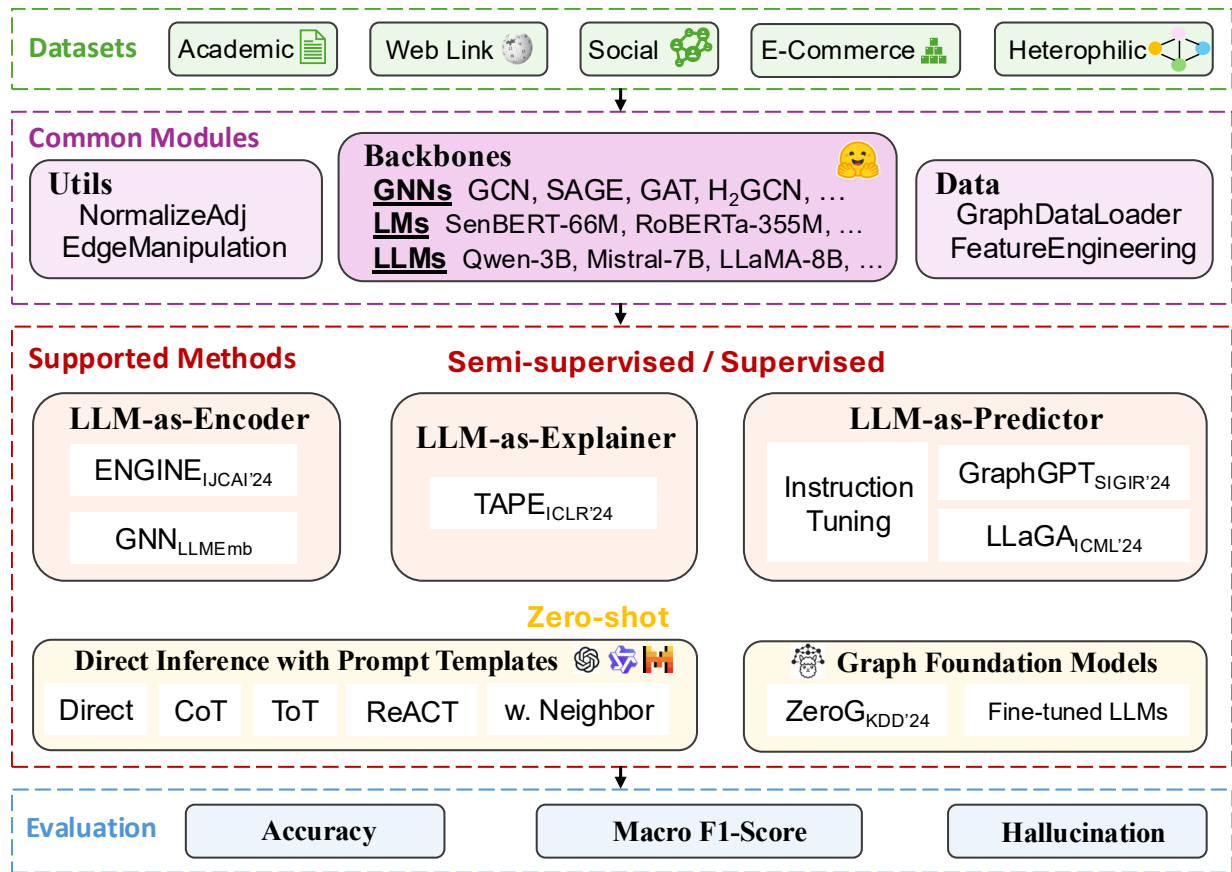


图 1: LLMNodeBed 总览（原文 Figure 1，图内英文为原文保留）。自上而下分五层：**Datasets** 数据集——覆盖学术、网页、社交、电商、异质图五大领域；**Backbones** 骨干模型——GNN (GCN/SAGE/GAT/H₂GCN)、小型语言模型 LM (SenBERT/RoBERTa)、大模型 LLM (Qwen/Mistral/LLaMA)；**Supported Methods** 支持的方法——半监督/监督下分 *LLM-as-Encoder*、*LLM-as-Explainer*、*LLM-as-Predictor* 三派，零样本下分带提示词的直接推理与图基础模型 *GFM*；**Evaluation** 评测——准确率 Accuracy、Macro-F1、幻觉率 Hallucination。统一骨干 + 统一代码，是“公平对比”的基础。

“公平”的关键设计：在主对比里，所有方法都用同一套组件——GNN 一律用 GCN，小模型 LM 一律用 RoBERTa-355M，大模型 LLM 一律用 Mistral-7B。这样性能差异就只来自方法设计本身，而不是“谁用了更强的骨干”。每个实验跑 4 次取平均。

数据集（源自原文 Table 1，节选关键列）：作者特意挑了同质性差异巨大的数据集，好观察“同质性”这个变量的影响。

表 1: 部分数据集统计（数值源自原文 Table 1）。同质率越低，图结构越“不可靠”。

	Cora	Pubmed	WikiCS	Reddit	Computer	Texas
领域	学术	学术	网页	社交	电商	网页 (异质)
节点数	2,708	19,717	11,701	33,434	87,229	187
同质率 (%)	82.52	79.24	68.67	55.52	85.28	6.69

6 六、统一分析框架：用“互信息”解释为什么

这篇论文虽然是实证分析，但它给出了一个贯穿全文的理论视角来解释各种现象——互信息 (mutual information) 分解。这是理解全文结论的“总钥匙”，值得专门讲一节。

节点分类（用交叉熵损失训练）本质上是在最大化“图 G ”与“标签 Y_l ”之间的互信息 $I(G; Y_l)$ 。作者把图 G 看成“节点属性 X （文字）”和“结构 E （边）”的联合分布，于是这个互信息可以拆成两项：

$$I(G; Y_l) = I(X, E; Y_l) = \underbrace{I(E; Y_l)}_{\text{结构提供的信息}} + \underbrace{I(X; Y_l | E)}_{\text{文字额外提供的信息}} . \quad (1)$$

【小白补丁】这个公式怎么读、有什么用

互信息 $I(A; B)$ 衡量“知道 A 能帮你确定 B 多少”，值越大越相关。公式 (1) 把“做对分类需要的总信息”拆成两块：

- 第一项 $I(E; Y_l)$ ：光靠图的结构（边）能提供多少线索——这是传统 GNN 吃的饭。同质图里这一项很大（沿边传播就够）。
- 第二项 $I(X; Y_l | E)$ ：在已知结构之后，节点文字还能额外补多少线索——这是 LLM 吃的饭。

一句话推论：当第一项（结构）已经很大时，第二项（文字）能发挥的空间就小，LLM 帮不上太多；当第一项很小时（如异质图、或标签很少导致学不到结构规律），第二项就成了主力，LLM 价值凸显。后面几乎所有结论，都能用这把钥匙解释。

举两个例子帮助体会：

- **为什么半监督下 LLM 更有用？** 标签极少时，模型很难从稀疏标签里学到可靠的“结构规律”，即 $I(E; Y_l)$ 实际可利用得很低，于是文字项 $I(X; Y_l | E)$ 成了救命稻草——LLM 的语义理解大显身手。
- **为什么监督下强 LLM 优势缩小？** 标签充足时 $I(E; Y_l)$ 被充分利用、变得很大，对文字的依赖下降，于是“用 GPT-4o 还是用小一点的模型”差别就不明显了。

7 七、关键发现与结果表解读（8 条 Takeaway）

这是本文精华。作者把 8 条洞见嵌在实验里，下面按“何时有用 / 何时无用”重新组织，并配关键数字。

7.1 第一组：半监督 vs. 监督——LLM 到底什么时候值

主结果 (数值源自原文 Table 2, 准确率%, 节选关键方法 + 平均分 Avg.):

表 2: 半监督与监督设定下的性能 (源自原文 Table 2, 准确率%)。Avg. 为多数数据集平均; 越高越好。

设定	代表方法 (类别)	半监督 Avg.	监督 Avg.
经典	GAT (纯 GNN, 浅层嵌入)	74.67	78.70
经典	RoBERTa (纯文本 LM)	68.06	78.17
LLM-as-Encoder	GCN _{LLMEmb}	78.51	81.09
LLM-as-Encoder	ENGINE	78.40	81.32
LLM-as-Explainer	TAPE	78.43	81.80
LLM-as-Predictor	LLaGA	75.07	81.69
LLM-as-Predictor	LLM _{IT} (纯微调 LLM)	65.76	72.38

怎么读这张表 (对应 Takeaway 1–5):

- **T1: 合理引入 LLM 总能提升。**最好成绩几乎都来自 LLM 方法, 且这一结论在百万级大图 (ogbn-products) 上同样成立——用 LLM 挖掘文字信息确实有用。
- **T2: 半监督的提升远大于监督。**从经典 GAT 到 LLM 方法, 半监督下 Avg. 从 74.7 涨到约 78.5 (提升明显); 而监督下大家都挤在 81 至 82 分, **LLM 相对经典 GNN 的优势变得微弱。**(这正是公式 (1) 的推论。)
- **T3: 标签强依赖文字时, Explainer (TAPE) 最猛。**在 Cora、WikiCS 等“结构没那么重要”的学术/网页数据上 TAPE 拿第一/第二; 但在需要深层结构理解的社交网络 (如 Reddit 上判断高热度用户) 就吃力。
- **T4: Encoder 是“性价比之王”。**LLM-as-Encoder 在所有数据集上都表现稳健, 且耗时远小于要“生成长解释”的 Explainer, 在效率与精度间取得平衡。
- **T5: Predictor 要靠“标签多”才行。**监督下 LLaGA 很强 (10 个数据集里 5 个最佳), 但半监督下 Predictor 不稳定 (Macro-F1 低、输出类别分布失衡)。尤其**纯微调 LLM (LLM_{IT}) 最弱且最慢** (arXiv 上一轮要 30+ 小时), 说明光有 LLM 不行, 必须把图结构喂进去。

7.2 第二组：零样本——不给标签时谁最强

零样本结果 (数值源自原文 Table 3, 准确率%, 4 数据集平均 Avg.):

表 3: 零样本设定下的准确率 (源自原文 Table 3, %, 4 数据集平均)。

类型	方法	Avg. Acc
强 LLM (GPT-4o)	Direct 直接提示	61.30
强 LLM (GPT-4o)	w. Summary (先总结邻居)	64.99
开源 LLM (LLaMA-8B)	Direct	49.56
开源 LLM (LLaMA-8B)	w. Summary	53.27
GFM 图基础模型	ZeroG	55.56
GFM (迁移的 Predictor)	LLaGA	28.53

怎么读 (对应 Takeaway 6–7):

- **T6: GFM 能赢开源 LLM, 但赢不了 GPT-4o。** ZeroG (55.6) 大体优于 LLaMA-8B (约 50), 却明显不及 GPT-4o (约 61 至 65)。而把训练好的 Predictor (LLaGA) 拿去跨数据集迁移时**彻底崩盘** (仅 28.5), 因为它的“投影器”是数据集专用的, 换个图就失灵。
- **T7: 给 LLM 喂结构信息能涨分, 但“花式提示词”几乎没用。** CoT、ToT、ReAct 这些 通用推理提示词对节点分类提升很小 (LLaMA 遇到复杂提示甚至会输出超长、难解析的内容); 真正有效的是**把邻居信息塞进去**——“w. Neighbor”尤其是“**w. Summary**” (先让 LLM 总结邻居再分类), 在同质图 (Cora、Photo) 上能涨 3% 到 10%。原因还是同质性: 邻居和自己同类, 邻居信息就是强线索。

7.3 第三组: 每一派内部的精细分析

T8: 图越“没用”, LLM 当编码器优势越大 (源自原文 Table 4–5)。对比“用 LLM 编码 vs. 用小模型 LM 编码”, 在同质图上二者差距很小; 但在**异质图**上, LLM 编码器优势巨大——例如 Texas 数据集上, 配合 GCN/SAGE/H2GCN, LLM 编码比 LM 编码**准确率高出约 12%**, 跨方法差距可达 10%。再次印证: 结构帮不上忙时 (异质图), 就得靠文字, 而 LLM 最会读文字。

Explainer 用强 LLM 划算吗? (Table 6–7): 把 TAPE 里的 Mistral-7B 换成 GPT-4o, 半监督下确有提升 (Pubmed 上 +4%), 但监督下提升微乎其微。**建议: 标签充足时, 用便宜的开源 LLM 当 Explainer 即可, 没必要为 GPT-4o 多花钱。**

Predictor 选多大的模型? (Table 8–9): 在 LLaGA 上把 Qwen 从 3B 加到 32B, 性能一开始随规模上升, 但过了 7B/14B 就基本饱和, 而 32B 推理慢 5 倍 (每样本 200+ 毫秒)。**建议: Predictor 选 Qwen-7B 或 14B 最划算; 同规模里 Mistral-7B 综合最优, 是推荐骨干。**

7.4 别忽视的两个工程现实：成本与幻觉

【小白补丁】成本（源自原文成本分析）：LLM 方法普遍又慢又费显存

经典方法极快：GNN 几秒收敛、LM 微调几分钟，推理可在毫秒级处理上千样本，适合工业实时部署。而 LLM 方法：Explainer 最慢（TAPE 在 Pubmed 上要 5.9 小时，因要给每个节点生成解释文本）；Encoder 在 LLM 方法里最快（十几分钟）且显存与普通 GNN 相当；Predictor 要数小时训练。**推理时 LLM 方法与经典方法仍有巨大效率鸿沟**——这正是选型时必须权衡的“性价比”。

【小白补丁】幻觉（hallucination）：Predictor 会“答非所问”

让 LLM 直接当 Predictor 时，它可能输出**标签集合里根本不存在的类别**（即幻觉）。论文附录专门统计了各 Predictor 方法的平均幻觉率，并展示了 GraphGPT 在 Citeseer/Pubmed 上的幻觉例子。这是“让 LLM 直接出标签”这条路独有的风险，Encoder/Explainer 路线因为最后用的是 GNN 分类头，不存在此问题。

8 八、关键结论速查表（何时有用 / 何时无用）

把全文 8 条洞见浓缩成一张“选型速查表”：

表 4: LLM 节点分类“何时用、用哪招”速查（本导读据原文 8 条 Takeaway 重排）。

你的场景	推荐做法	为什么 / 注意
标签很少（半监督）	上 LLM, 优先 Encoder 或 Explainer(TAPE)	提升最明显；文字信息此时最值钱
标签很多（监督）	经典 GNN 通常就够	LLM 优势微弱, 未必抵得过成本
异质图 / 结构没用	LLM-as-Encoder	比小模型 LM 高出可达 12%，优势最大
要均衡精度与成本	LLM-as-Encoder	LLM 里最快、显存最省，性价比之王
标签足 + 想直接出标签	LLaGA (Predictor)	强但要喂图结构；纯微调 LLM 最差最慢
零样本、没法训练	强 LLM(GPT-4o) + 邻居/总结提示	比 GFM 更好；花式推理提示几乎无用
零样本且想用 GFM	ZeroG	胜开源 LLM、不及 GPT-4o；迁移型 Predictor 会崩
Explainer 选骨干	标签足时用 便宜开源 LLM 即可	换 GPT-4o 在监督下几乎不涨分
Predictor 选规模	Qwen-7B/14B 最划算	过了就饱和，32B 慢 5 倍

9 九、小白知识点卡片（速查）

- **节点分类 (Node Classification)**: 给图里的点打标签的任务。
- **文本属性图 / TAG**: 每个节点都带一段文字的图。
- **同质性 (Homophily)**: 相连节点同类的概率；高 = 结构有用，低 = 要靠文字。
- **LLM-as-Encoder**: 让 LLM 把文字编码成好特征，再交给 GNN 分类。
- **LLM-as-Explainer**: 让 LLM 先把节点文字“扩写补充”，增强后再分类（如 TAPE）。
- **LLM-as-Predictor**: 把图喂进 LLM，让它直接输出标签（如 GraphGPT、LLaGA）。
- **GFM (图基础模型)**: 在大图上预训练、可迁移到新图的模型（如 ZeroG）。

- **半监督 / 监督 / 零样本**：分别是“极少标签 / 大量标签 / 无标签训练”。
- **互信息 $I(\cdot;\cdot)$** ：衡量两个量相关程度；本文用它解释“结构信息 vs. 文字信息”谁主导。
- **幻觉 (Hallucination)**：模型输出不存在的类别/内容，是 Predictor 路线的风险。
- **Macro-F1**：对各类别一视同仁的 F1 平均，能暴露“类别不平衡时”的虚高准确率。
- **CoT / ToT / ReAct**：思维链/思维树/推理-行动等通用提示词技巧。

10 十、一句话总结与谱系定位

一句话：这篇论文不造新模型，而是搭了个统一擂台 LLMNodeBed、跑了 2,700+ 个模型，用一把“互信息分解”的钥匙把结论讲透——**标签少 / 图没用 (异质) 时该上 LLM (尤其当 Encoder)**，**标签多时经典 GNN 就够**；**零样本时把图结构喂给强 LLM 直接推理最香**。

在“图 × 大模型”谱系里的位置：本系列里大多数论文是在提出一种新方法——G-Retriever 做图上 RAG、LLaGA 做投影对齐、GraphGPT/InstructGraph 做指令微调、G1 做强化学习……而本文是一篇“元层面”的分析/基准工作：它把上述各类方法 (Encoder/Explainer/Predictor/GFM) **拉到同一标准下公平横评**，回答“到底该选谁”。因此它最适合作为**工程选型手册**与**入门地图**：读完它，你再去看具体某个方法的论文，就知道那个方法属于哪一派、强在什么设定下。

动手复现：官方代码与数据 <https://llmnodebed.github.io/>（一条命令复现全部表格），是系统了解“LLM 做图节点分类”全景的极佳起点。